

UniFECAF
Análise e Desenvolvimento de Sistemas

SISTEMA AUTOMOTIVO

Gestão de Estoque de Veículos

Disciplina: Programação Orientada a Objetos

Tecnologias: Java · Spring Boot · MySQL · REST API

Relatório Teórico — Parte 1 da Entrega

Sumário

1. Levantamento de Requisitos (10 perguntas com simulação de respostas)
2. Descrição das Entidades do Sistema
3. Aplicação dos Conceitos de POO
4. Requisitos Funcionais e Não Funcionais
5. Justificativa Tecnológica

1. Levantamento de Requisitos

Para definir corretamente o escopo e as funcionalidades do sistema, foi realizada uma simulação de entrevista com o cliente — neste caso, o gerente de uma concessionária de veículos. As 10 perguntas abaixo representam o processo de elicitação de requisitos, técnica fundamental da Engenharia de Software para alinhar as expectativas do cliente com a solução a ser desenvolvida.

P1: Quais informações são essenciais para cadastrar um veículo no estoque?

R: Precisamos registrar a marca, o modelo, o ano de fabricação, a cor, o preço de venda, a quilometragem atual e se o veículo está disponível, vendido ou reservado. Às vezes também precisamos de um campo de observações para anotações internas.

P2: Como vocês organizam as marcas e modelos hoje? Existe uma hierarquia entre eles?

R: Sim. Uma Marca (ex: Toyota) pode ter vários Modelos (ex: Corolla, Hilux). E um Modelo pode ter várias unidades no estoque com cores e anos diferentes. Precisamos dessa estrutura para facilitar a busca.

P3: Quais filtros de busca são mais utilizados pela equipe de vendas no dia a dia?

R: Os vendedores filtram principalmente por marca, faixa de preço e ano do veículo. Os clientes costumam chegar com um orçamento definido, então o filtro de preço máximo é o mais importante.

P4: Com que frequência as informações de um veículo precisam ser atualizadas?

R: O status muda bastante — um carro pode ser 'disponível', depois 'reservado' e por fim 'vendido' em questão de dias. O preço também é ajustado periodicamente. A quilometragem pode mudar se o carro for testado por um cliente.

P5: O que deve acontecer quando um veículo é vendido? Ele é excluído ou apenas marcado?

R: Por questão de histórico, prefiro que ele seja marcado como 'Vendido' e não deletado. Mas precisamos de uma opção de remoção definitiva para casos de cadastros errados ou veículos descontinuados da frota.

P6: Quantos usuários vão utilizar o sistema simultaneamente? Quais são seus perfis?

R: Temos em média 5 vendedores e 2 administradores. Os vendedores precisam consultar e atualizar status, enquanto os administradores devem ter controle total, incluindo cadastro de novas marcas, modelos e remoção de registros.

P7: O sistema precisa se integrar com alguma outra ferramenta existente (ERP, site, etc.)?

R: No futuro sim, mas por agora o mais importante é ter a API funcionando. Queremos expor uma lista de veículos disponíveis para o nosso site futuramente, então uma API REST seria ideal.

P8: Há alguma regra de negócio específica? Por exemplo, restrição de preço mínimo ou validação de ano de fabricação?

R: O ano de fabricação não pode ser inferior a 1990 nem superior ao ano corrente + 1 (para veículos 0km do próximo ano). O preço não pode ser zero ou negativo. Veículos com status 'Vendido' não devem aparecer nas consultas públicas.

P9: Existe necessidade de relatórios ou exportação de dados (ex: lista de estoque em PDF/Excel)?

R: Seria ótimo ter uma listagem completa do estoque disponível para imprimir ou enviar por e-mail. Por enquanto, uma listagem via sistema já resolve. Relatórios mais completos podem ser uma evolução futura.

P10: Qual é a expectativa de performance? Existe um grande volume de veículos cadastrados?

R: Trabalhamos com um estoque de 50 a 300 veículos normalmente. O sistema precisa retornar resultados de consulta em menos de 2 segundos. Não é um volume muito alto, mas a resposta precisa ser ágil para não atrasar o atendimento ao cliente.

2. Descrição das Entidades do Sistema

O sistema é modelado em torno de três entidades principais que se relacionam entre si, formando a estrutura de dados do domínio automotivo. Abaixo, cada entidade é descrita com seus atributos, responsabilidades e relacionamentos.

2.1 Entidade: Marca

Representa o fabricante do veículo (ex: Toyota, Ford, Honda). É a entidade de mais alto nível na hierarquia do domínio. Uma Marca pode estar associada a múltiplos Modelos.

Atributo	Tipo	Descrição	Restrições
id	Long	Identificador único (PK)	Auto-gerado, Not Null
nome	String	Nome da fabricante	Not Null, Único, Max 100
paisOrigem	String	País de origem da marca	Max 50
modelos	List<Modelo>	Modelos vinculados	Relacionamento 1:N

2.2 Entidade: Modelo

Representa uma linha de produto de um fabricante (ex: Corolla, Civic, Onix). Um Modelo pertence a uma única Marca (relacionamento N:1) e pode ter múltiplas unidades de Veiculo no estoque.

Atributo	Tipo	Descrição	Restrições
id	Long	Identificador único (PK)	Auto-gerado, Not Null
nome	String	Nome do modelo	Not Null, Max 100
categoria	String	Categoria (SUV, Sedan, etc.)	Max 50
marca	Marca	Marca proprietária	FK para Marca, Not Null
veiculos	List<Veiculo>	Unidades em estoque	Relacionamento 1:N

2.3 Entidade: Veiculo

É a entidade central e mais completa do sistema. Representa uma unidade física específica no estoque, com suas características individuais. Cada Veiculo pertence a um Modelo.

Atributo	Tipo	Descrição	Restrições
id	Long	Identificador único (PK)	Auto-gerado, Not Null
anoFabricacao	Integer	Ano de fabricação	Not Null, >= 1990
cor	String	Cor da carroceria	Not Null, Max 50

preco	BigDecimal	Preço de venda	Not Null, > 0
quilometragem	Integer	Km rodados	Not Null, >= 0
status	StatusVeiculo	Situação no estoque	DISPONIVEL/VENDIDO/RESERVADO
observacoes	String	Anotações internas	Opcional, Max 500
modelo	Modelo	Modelo do veículo	FK para Modelo, Not Null
dataCadastro	LocalDate	Data de inserção	Auto-preenchido

Diagrama de Relacionamento: Marca (1) → (N) Modelo (1) → (N) Veiculo

3. Aplicação dos Conceitos de POO

A Programação Orientada a Objetos (POO) é o paradigma central deste projeto. Abaixo, cada pilar da POO é explicado e exemplificado com trechos do sistema desenvolvido.

3.1 Classes e Atributos

Uma Classe é um molde que define o comportamento e os dados de objetos. No projeto, cada entidade de negócio foi modelada como uma classe Java. Os Atributos são as variáveis que armazenam o estado do objeto. Por exemplo, a classe *Veiculo* possui atributos como *preco*, *cor* e *quilometragem*, que descrevem uma unidade específica no estoque.

```
// Exemplo de declaração de classe e atributos @Entity public class Veiculo {
@Id @GeneratedValue private Long id; private String cor; private BigDecimal
preco; @Enumerated(EnumType.STRING) private StatusVeiculo status; }
```

3.2 Encapsulamento

O Encapsulamento protege os dados internos de uma classe, expondo apenas o que é necessário através de métodos públicos (getters e setters). No projeto, todos os atributos das entidades são declarados como **private**, e o acesso é controlado via métodos anotados com *@Getter* e *@Setter* da biblioteca Lombok, garantindo que nenhuma classe externa manipule os dados diretamente sem passar pelas regras de negócio da camada de Serviço.

```
// Encapsulamento com Lombok @Entity @Getter @Setter @NoArgsConstructor public
class Marca { @Id @GeneratedValue private Long id; // Atributo privado private
String nome; // Acesso apenas por getNome() / setNome() }
```

3.3 Herança

A Herança permite que uma classe filha herde atributos e métodos de uma classe pai, promovendo o reuso de código. No projeto, criamos uma classe abstrata *VeiculoBase* que contém os atributos e comportamentos comuns a todos os tipos de veículos. As classes filhas *VeiculoNovo* e *VeiculoUsado* herdam esses comportamentos e adicionam suas particularidades (ex: *VeiculoNovo* tem *garantiaMeses*, enquanto *VeiculoUsado* tem *laudoVistoria*).

```
// Herança: classe pai abstrata public abstract class VeiculoBase { protected
Long id; protected String cor; protected BigDecimal preco; public abstract
String getTipoVeiculo(); } // Classe filha para veículos novos public class
VeiculoNovo extends VeiculoBase { private int garantiaMeses; @Override public
String getTipoVeiculo() { return "NOVO"; } } // Classe filha para veículos
usados public class VeiculoUsado extends VeiculoBase { private boolean
laudoVistoria; @Override public String getTipoVeiculo() { return "USADO"; } }
```

3.4 Polimorfismo

O Polimorfismo permite que objetos de diferentes classes sejam tratados de forma uniforme por meio de uma interface ou classe pai. No exemplo acima, um método que recebe um *VeiculoBase* pode chamar *getTipoVeiculo()* sem saber se é um *VeiculoNovo* ou *VeiculoUsado* — cada um responderá de acordo com sua própria implementação. Isso é fundamental para a extensibilidade do sistema, pois novos tipos de veículos podem ser adicionados sem alterar o código existente.

3.5 Abstração

A Abstração consiste em esconder os detalhes de implementação e expor apenas o que é relevante para o usuário do componente. No projeto, a camada de Service abstrai toda a complexidade das operações de banco de dados: o Controller apenas chama *veiculoService.findAll(filtros)* sem saber como a query é construída internamente. As interfaces de Repository (Spring Data JPA) são outro exemplo: o desenvolvedor chama *veiculoRepository.findByMarcaId(id)* sem escrever uma linha de SQL.

4. Requisitos Funcionais e Não Funcionais

4.1 Requisitos Funcionais

Descrevem O QUE o sistema deve fazer — as funcionalidades entregues ao usuário final.

ID	Requisito	Descrição
RF01	Cadastro de Veículos	O sistema deve permitir inserir um veículo com: modelo, marca, ano, cor, preço, quilometragem
RF02	Cadastro de Marcas	O sistema deve permitir inserir, editar e remover marcas de fabricantes.
RF03	Cadastro de Modelos	O sistema deve permitir inserir modelos vinculados a uma marca existente.
RF04	Consulta de Veículos	O sistema deve listar todos os veículos com suporte à paginação.
RF05	Filtro por Marca	O sistema deve permitir filtrar o estoque pelo nome ou ID da marca.
RF06	Filtro por Preço	O sistema deve permitir busca por faixa de preço (preçoMin e preçoMax).
RF07	Filtro por Ano	O sistema deve permitir filtrar veículos por ano de fabricação.
RF08	Filtro por Status	O sistema deve retornar apenas veículos com determinado status (DISPONIVEL, VENDIDO, RE
RF09	Atualização de Veículo	O sistema deve permitir editar preço, quilometragem, status e observações de um veículo.
RF10	Remoção de Veículo	O sistema deve permitir excluir um veículo do banco de dados (hard delete).

4.2 Requisitos Não Funcionais

Descrevem COMO o sistema deve se comportar — qualidade, restrições e características técnicas.

ID	Categoria	Descrição
RNF01	Performance	Consultas e filtros devem retornar resultados em menos de 2 segundos para até 500 registros.
RNF02	Usabilidade	A API deve seguir os padrões REST, com retornos em JSON e códigos HTTP adequados (200, 201
RNF03	Escalabilidade	A arquitetura em camadas (Controller → Service → Repository) deve facilitar a adição de novos rec
RNF04	Segurança	Parâmetros de entrada devem ser validados com Bean Validation (@NotNull, @Min, @Size) para ev
RNF05	Manutenibilidade	O código deve seguir as convenções Java e o padrão de nomenclatura camelCase, com comentáric
RNF06	Portabilidade	O sistema deve rodar em qualquer ambiente com Java 17+ e MySQL 8+, configurável via applicatio

5. Justificativa Tecnológica

A escolha das tecnologias foi fundamentada nos requisitos do projeto, na maturidade das ferramentas e na sua adoção no mercado de trabalho brasileiro.

5.1 Java 17

Java é uma das linguagens mais utilizadas no mercado corporativo brasileiro, especialmente em sistemas de médio e grande porte. Sua tipagem estática e orientação a objetos nativa alinham-se diretamente com os requisitos da disciplina. A versão 17 (LTS) garante suporte de longo prazo, com recursos modernos como Records e melhorias de performance na JVM. A vasta documentação e a grande comunidade facilitam a resolução de problemas durante o desenvolvimento.

5.2 Spring Boot 3.x

Spring Boot é o framework Java mais popular para construção de APIs REST. Ele elimina a necessidade de configurações complexas de XML (como no Spring MVC tradicional), permitindo criar um servidor HTTP funcional com poucas linhas de código. Seus módulos — Spring Web (endpoints REST), Spring Data JPA (persistência) e Spring Validation (validações) — cobrem todos os requisitos deste projeto de forma integrada e padronizada. O padrão arquitetural MVC (Model-View-Controller), implementado naturalmente pelo Spring, separa responsabilidades e facilita a manutenção e os testes.

5.3 MySQL 8

MySQL é o banco de dados relacional open-source mais utilizado no mundo, com ampla adoção em aplicações web e sistemas corporativos. Sua escolha é justificada pela natureza relacional dos dados do projeto: os relacionamentos entre Marca, Modelo e Veículo são melhor representados em um banco relacional, com integridade referencial garantida por chaves estrangeiras. O MySQL 8 oferece suporte a transações ACID, índices eficientes para as queries de filtro e integração nativa com o Hibernate (ORM do Spring Data JPA) via o driver JDBC.